

Crowd Counting and Zone Management System

Internship Project Report

Author: Bangaru Balu

Yogendra Reddy

Mentor: Umme Asma

Training Dates: 22 September 2025 – 21 November 2025

Abstract

This report introduces a "**Crowd Counting and Zone Management System**". It's an automatic tool designed to count, track, and watch people in real-time, making monitoring crowds much faster and more accurate than manual supervision.

The system uses advanced computer vision technology, specifically the **YOLOv8** model, along with **OpenCV**, to instantly recognize and follow every person in a video feed. A main safety feature is the ability to draw custom "**Danger Zones**" on the screen. The system then automatically tracks how long each person stays (**dwell time**) inside these zones.

We built the system using **React** for the simple user-friendly website and **Django** for the powerful backend. **Redis** is used to store data quickly and speed up communication. All critical data, like alerts and user information, is kept securely with end-end encryption.

If a person stays too long in a danger zone, too many people gather in one area, or the crowd gets too dense, the system immediately triggers a three-level **alert mechanism**. The website features a secure, **role-based login** (Admin and User), a live dashboard showing real-time data with charts, and an admin panel for easy setup. Finally, the system can create downloadable **CSV reports** for reviewing past incidents.

Keywords: Crowd Monitoring, Computer Vision, YOLOv8, Django, Redis server, Real-time Analytics, Object Tracking, React.

Acknowledgements

I would like to express my sincere gratitude to my mentor, **Umme Asma**, for her invaluable guidance, support, and encouragement throughout this internship project. Her insights and feedback were instrumental in the successful development and completion of this system.

I am also thankful to the **Infosys Springboard** program for providing the opportunity and resources to work on this challenging and meaningful project.

Contents

Abstract	1
Acknowledgements	2
Contents	3
List of Figures	5
List of Tables	6
1 Introduction	7
1.1 Project Background and Motivation	7
1.2 Problem Statement	7
1.3 Project Objectives	8
1.4 Report Structure	8
2 System Design and Technology	10
2.1 System Architecture	10
2.2 Technical Workflow	11
2.3 Technology Stack Rationale	12
3 Implementation Details	14
3.1 Core Software Modules	14
3.2 Database Schema	15
3.2.1 User Table	15
3.2.2 Alert History Table	16
3.2.3 System Settings Table	16
4 Results and System Functionality	18
4.1 User Authentication	18
4.2 Analysis and Live Dashboard	18
4.3 Data Visualization and Reporting	20
4.4 Admin Panel and System Configuration	20
5 Challenges and Future Scope	24
5.1 Challenges Encountered	24
5.2 Future Enhancements	24
6 Conclusion	26

Bibliography	27
Bibliography	27
Appendices	28
A Setup and Installation Guide	29
A.1 Prerequisites.....	29
A.2 Installation Steps	29
A.3 Running the Application.....	30
A.4 Project Dependencies (requirement.txt)	30

List of Figures

2.1	High-Level System Architecture	11
3.1	Database Entity-Relationship Diagram	16
4.1	User Login Page	18
4.2	User Registration Page.....	19
4.3	New Analysis Selection Page.....	19
4.4	Live Dashboard Overview	20
4.5	Live Summary Dashboard with Charts	21
4.6	Alert History Page with CSV Download	22
4.7	Admin Panel for System Configuration.....	23

List of Tables

2.1	Technology Stack Rationale	12
3.1	Database Schema Details	17

Chapter 1

Introduction

1.1 Project Background and Motivation

Watching crowds in busy places like malls, airports, and stadiums is usually done by **security guards manually watching video screens**. This traditional method is **slow, often inaccurate, and cannot provide real-time data**. In very crowded areas, even a small delay in noticing a safety risk (like overcrowding or someone entering a restricted zone) can have dangerous consequences. However, thanks to new, powerful **AI and computer vision tools**, we can now automate this process. Our goal was to create an intelligent system that can constantly monitor multiple video feeds, measure crowd behavior accurately, and give **immediate alerts**. This shifts security from *reacting* to incidents after they happen to **proactively preventing** them

1.2 Problem Statement

Watching crowds by hand has major limitations. It is **not practical for large spaces**, the human eye gets **tired**, and it's always **subjective** (it depends on what the person thinks). This leads to four main issues:

- **Slow Response:** Security staff cannot watch every area at the same time. This means they are often **slow to react** to too many people gathering or to security issues.
- **Wrong Numbers:** Counting people by hand is easy to get wrong. This makes it impossible to collect **accurate data** needed for planning or meeting safety rules.
- **No Insightful Data:** Manual systems cannot easily measure important details, like **how long people stay** in one area (dwell time), how people move through the space, or what has happened over time. This data is essential for better safety rules and space planning.
- **Very Expensive:** To monitor crowds effectively by hand, you need to hire many security people, which results in **high operating costs**.

The Solution: We clearly need a system that is **automatic, accurate, and smart**. It must be able to detect, track, and count people in real-time, watch specific danger zones, and send customized alerts to make places safer and run more efficiently.

1.3 Project Objectives

The primary goal of this project is to design and develop a comprehensive AI-Based Crowd Monitoring and Safety Management System. The specific objectives are:

- **Real-time People Counting:**

The system can count how many people are present at any moment using a CCTV or webcam feed. For example, if 50 people are walking in a mall entrance, the system will instantly show “50” on the screen and update as people move in or out.

- **Live Dashboard with Visuals:**

There will be an interactive dashboard that shows real-time information such as **current people count, movement patterns, and area-wise statistics**. It will also include **heatmaps**, which use colors to show which zones are crowded (for example, red for busy areas, green for less crowded ones). This makes it very easy to understand what’s happening at a glance.

- **Zone-wise Management:**

The system can divide the place into different zones — for example, Zone A (Entrance), Zone B (Shopping Area), Zone C (Food Court) — and display the number of people in each zone separately. This helps in monitoring which area is busier and which one is free.

Alerts for Overcrowding:

The system can send automatic alerts when a particular zone becomes too crowded. For example, if Zone B allows only 100 people and the count crosses 100, the system can **flash a warning message or send a notification** to staff to control the crowd.

- **Exportable Reports:**

The system can generate and export detailed reports that include daily or weekly people count, busiest time periods, and crowd trends. These reports can be saved as **PDF or Excel files** for record-keeping and analysis.

- **Time-based Analysis:**

The dashboard will also show how the crowd changes throughout the day — for example, showing that the morning hours are less crowded but evenings are the busiest. This helps

managers plan staff and resources better.

■ **User-friendly Interface:**

The system will be designed in a simple, easy-to-use format, so even people without technical knowledge (like mall managers or security staff) can use it comfortably.

1.4 Report Structure

This report is organized into the following chapters:

- **Chapter 1: Introduction** – Outlines the project’s background, problem statement, and objectives.
- **Chapter 2: System Design and Technology** – Details the high-level system architecture, technical workflow, and the rationale for the selected technology stack.
- **Chapter 3: Implementation Details** – Provides a low-level overview of the core software modules, the database schema, and the logic of the alert mechanism.
- **Chapter 4: Results and System Functionality** – Demonstrates the final application’s features, including user authentication, the live dashboard, admin panel, and reporting, with illustrative screenshots.
- **Chapter 5: Challenges and Future Scope** – Discusses the challenges faced during development and proposes potential enhancements for future work.
- **Chapter 6: Conclusion** – Summarizes the project’s achievements and its overall success in meeting the defined objectives.
- **Bibliography** – Lists the research papers and resources cited in this report.
- **Appendix** – Provides the complete setup and installation guide for reproducing the project environment.

Chapter 2

System Design and Technology

This chapter outlines the high-level architecture of the system, the step-by-step technical workflow, and the rationale behind the chosen technology stack.

2.1 System Architecture

The system is designed as a full-stack web application with a modular architecture, separating the video processing pipeline from the web server and database. The workflow, as shown in Figure 2.1, follows a clear data-processing path.

The workflow is as follows:

1. **Video Input:** The system ingests raw video frames from a file or live webcam feed.
2. **YOLOv8 Detection:** Each frame is processed by the YOLOv8 (yolov8n.pt) model, which detects all instances of the "person" class.
3. **Person Tracker (detector.py):** The detection bounding boxes are passed to a tracker (ByteTrack), which assigns and maintains a unique ID for each person. This module also calculates dwell time within the user-defined zone and checks for alert conditions.
4. **Django Backend (views.py):** This is the central controller. It serves the video stream, handles user authentication, and provides API endpoints. It logs alerts to the database and reads settings from it.
5. **Database (SQLite):** The database stores persistent data: user accounts, all generated alert logs, and system-wide settings.
6. **Frontend (Client):** The user's web browser interacts with the backend. It displays the live video stream, fetches data from the API to populate dashboard charts (Chart.js), and allows admins to configure settings.
7. **Report Generator:** A separate module (report_generator.py) is called by the Flask backend to generate PDF reports on demand.

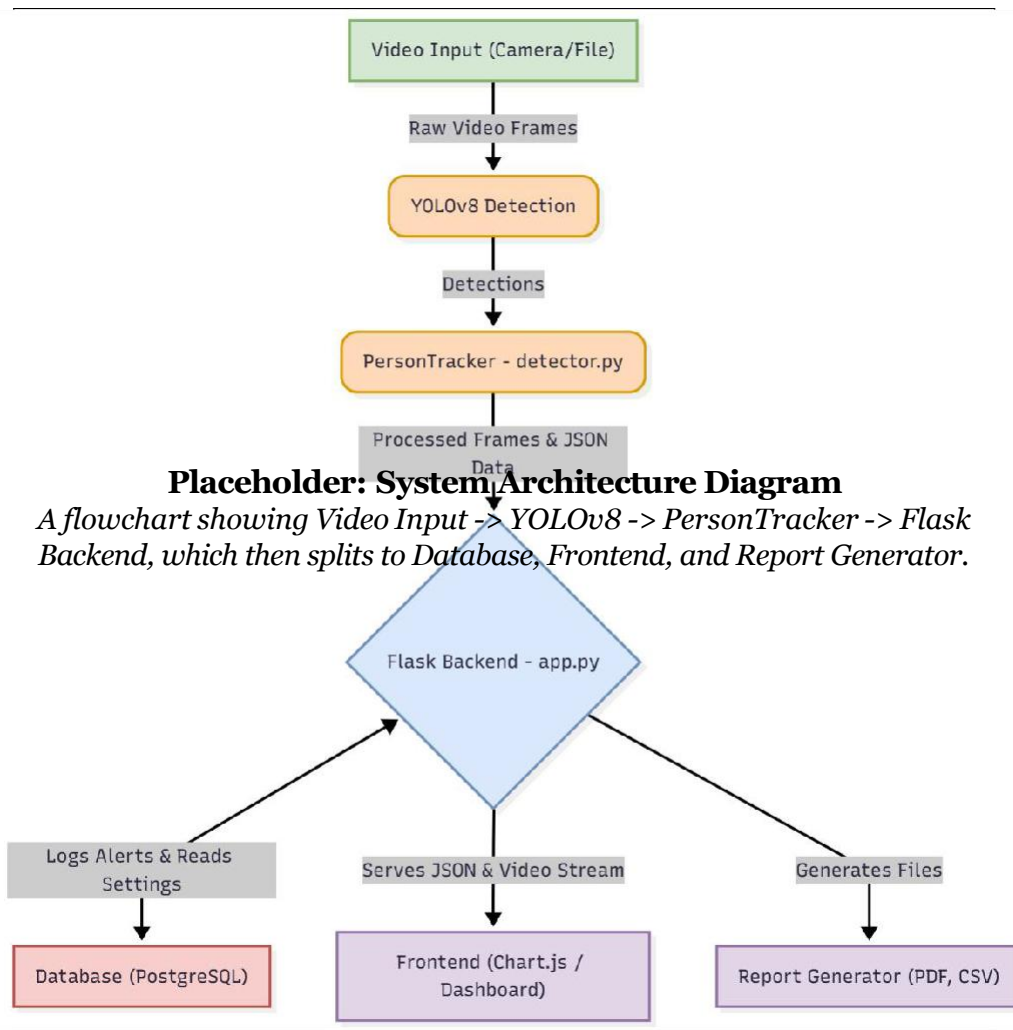


Figure 2.1: High-Level System Architecture

2.2 Technical Workflow

The operational flow from the user's perspective is as follows:

1. A user registers or logs into the web application. The Flask backend authenticates them using Flask-JWT-Extended and assigns a role ('user' or 'admin').
2. The user navigates to the analysis page and selects a video source (webcam or file upload).
3. An OpenCV window is presented, prompting the user to interactively draw a polygonal "Danger Zone" with their mouse.
4. Once the zone is defined, the detector.py module begins processing the video stream.
5. The YOLOv8 model, configured with a 'bytetrack.yaml' tracker, detects and tracks individuals, assigning persistent IDs (e.g., P1, P2).
6. For each frame, the system checks the position of each person (P1, P2, etc.) against the defined zone.

7. It calculates the cumulative time spent by each person inside the "Red Zone" versus the "Green Zone."
8. The system checks alert conditions against thresholds read from the SystemSettings table in the database.
9. If an alert is triggered (e.g., "P1 dwell time > 10s"), it is logged to the AlertHistory table in PostgreSQL and visually overlaid on the video stream.
10. The frontend dashboard continuously polls the /person_data API endpoint using AJAX.
11. This API returns a JSON object with real-time data, which JavaScript uses to update the Chart.js graphs (Zone Population, Total Population, etc.) without a page reload.
12. The user can download a PDF report for an individual or a CSV of their entire alert history. An admin can perform these actions for any user and can also modify the alert thresholds in the admin panel.

2.3 Technology Stack Rationale

The selection of technologies was crucial to balancing real-time performance, scalability, and development speed. The rationale for each component is detailed in Table 2.1.

Table 2.1: Technology Stack Rationale

Component	Technology	Purpose in This Project
Backend Framework	Django	The core web server handling routes, authentication (JWT), logic, and serving all data and video streams.
AI Detection	YOLOv8 (Ultralytics)	State-of-the-art real-time object detection model. The lightweight 'yolov8n.pt' model was used to find people in video frames with high speed.
AI Tracking	ByteTrack / BoSORT	The tracking algorithm (configured via YAML) used by YOLOv8 to assign and maintain persistent, unique IDs for each detected person.
Video Processing	OpenCV-Python	Used for capturing the video feed (from file or webcam), processing individual frames, drawing overlays (zones, boxes, text), and generating the MJPEG live stream.
Database	SQLite	A robust, scalable, and open-source relational database used to store all persistent data, including user credentials, alert logs, and system settings.

Table 2.1 – continued from previous page

Component	Technology	Purpose in This Project
Database ORM	Django-rest	Manages the database schema and handles all database queries in an object-oriented way within Python, improving code readability and security.
Authentication	JWT-Extended	Manages user sessions using JSON Web Tokens (JWT) for secure, stateless, and role-based access control to the dashboard and admin panel.
Frontend Logic	JavaScript (AJAX/Fetch)	Used to asynchronously fetch data (e.g., from the /person_data API) from the back-end to update the dashboard live without requiring a page reload.
Data Visualization	Chart.js	A JavaScript library used to create the responsive, live-updating charts (line, bar, scatter) on the user dashboard and admin panel.
PDF Reporting	Blob	A Python library used to dynamically generate and serve PDF reports summarizing an individual person's activity and alerts.

Chapter 3

Implementation Details

This chapter provides a low-level description of the project's core components, focusing on the software modules and the database schema that form the system's backbone.

3.1 Core Software Modules

The application logic is encapsulated in several key Python files and JavaScript files.

Views.py (Backend Controller) This is the main Flask application file. Its primary responsibilities include:

- Initializing the Django app, SQLite (database), and JWTManager (authentication).
- Defining all application routes (e.g., /login, /dashboard, /admin).
- Handling user registration, login, and logout logic.
- Protecting routes using @jwt_required and custom admin-only decorators.
- Defining API endpoints to serve data (/person_data) and manage the system (/admin/update_settings).
- Serving the live video stream as a multipart response (MJPEG).
- Handling report generation requests by calling the report generator and serving the resulting file.
- Defining the database models (User, AlertHistory).

consumer.py (AI Detection Engine) This module is the AI "brain" of the system, run as a separate process or thread managed by app.py.

- Initializes the YOLOv8 model with the specified tracker configuration.
- Contains the logic for capturing frames from the video source.
- Manages the interactive zone-drawing functionality using OpenCV's mouse callback functions.
- In its main processing loop, it passes frames to the YOLO model.
- Receives track data (bounding boxes and IDs) from the tracker.
- Maintains a dictionary to store track_data (e.g., dwell time) for each unique ID.

- Checks each person's location against the defined zone polygon.
- Increments dwell time and checks against the thresholds (fetched from the DB via `views.py`) to trigger alerts.
- Draws all visualizations on the frame (bounding boxes, zones, heatmaps, alert text) before yielding it to the web server.

Dashboard.jsx (PDF Generator) A utility module dedicated to creating PDF reports.

- Contains a function that accepts person-specific data (ID, zone times, alerts).
- Uses the ReportLab library to programmatically create a PDF document.
- Styles the document and formats the data into a clean table structure.
- Returns the generated PDF, which `app.py` then serves to the user.

script.jsx (User Dashboard Logic) This file controls the interactivity of the main user dashboard.

- Uses `fetch()` and `setInterval()` to continuously poll the `/person_data` API endpoint.
- Parses the incoming JSON response.
- Updates the live alert list, the person data table, and the `Chart.js` graphs with the new data.

admin.js (Admin Panel Logic) This file manages the UI and logic for the admin panel.

- Handles the tabbed interface for switching between User Management, Settings, and Logs.
- Populates the "Alerts Per User" chart.
- Manages the form submissions for updating system settings and user roles via AJAX, preventing page reloads.

3.2 Database Schema

The system's persistent data is stored in a SQLite database, managed by Django-SQLite. The schema is normalized and consists of three main tables, as detailed in Table 3.1 and illustrated in the ER diagram (Figure 3.1).

3.2.1 User Table

This table stores all user accounts, credentials, and access levels. The `password_hash` field stores passwords encrypted using `sha_256` algorithm. The `role` field is critical for access control, differentiating 'user' from 'admin'.

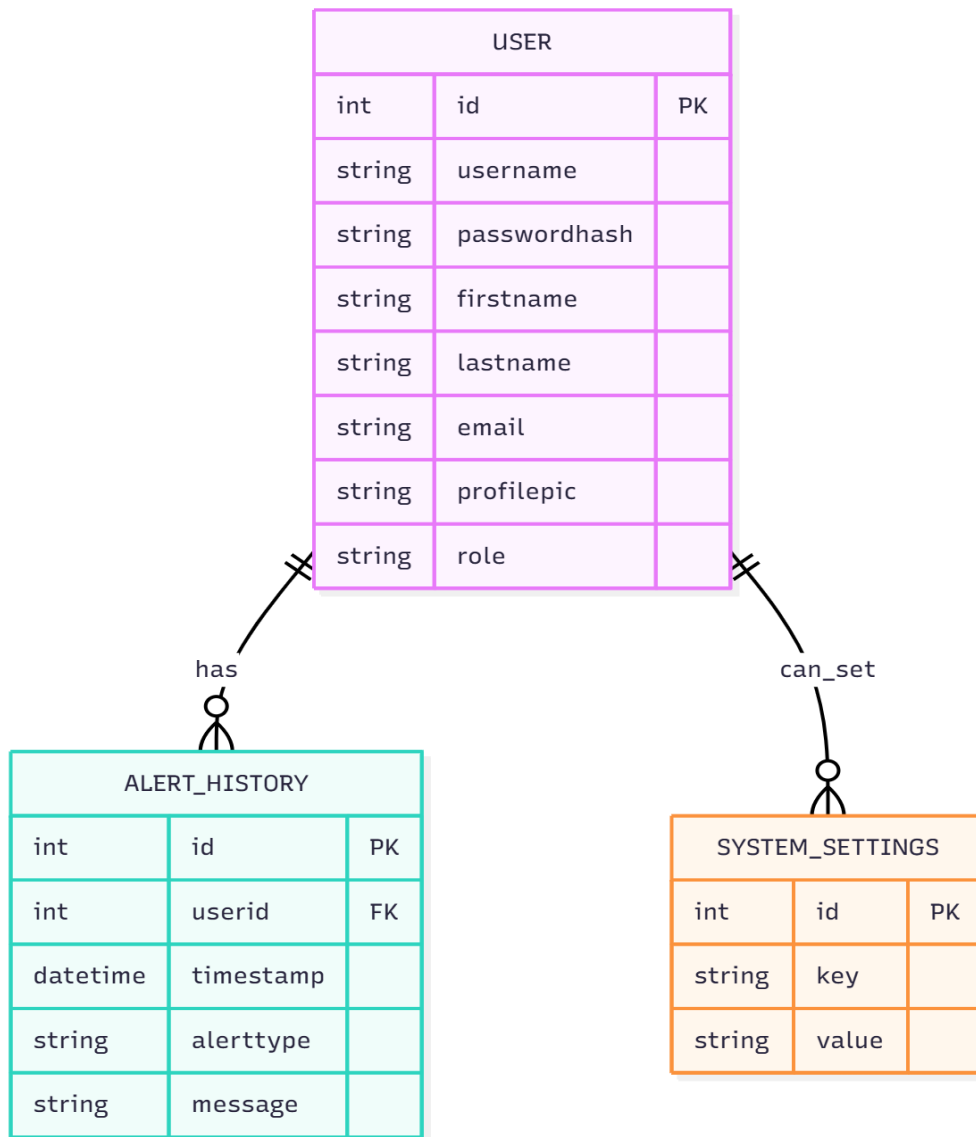


Figure 3.1: Database Entity-Relationship Diagram

3.2.2 AlertHistory Table

This table acts as a permanent log for every alert generated by the system. It is linked to the User table via the user_id foreign key, allowing for queries that retrieve all alerts generated under a specific user's session.

3.2.3 SystemSettings Table

This table is a key-value store for system-wide configuration, allowing for dynamic updates from the admin panel without restarting the server. It stores the thresholds that detector.py uses to trigger alerts. Default keys include:

- **person_threshold:** Max time (in seconds) a person can stay in the red zone. (e.g., 10)
- **zone_threshold:** Max number of people allowed in the red zone. (e.g., 5)
- **overall_threshold:** Max total people allowed in the entire frame. (e.g., 20)

Table 3.1: Database Schema Details

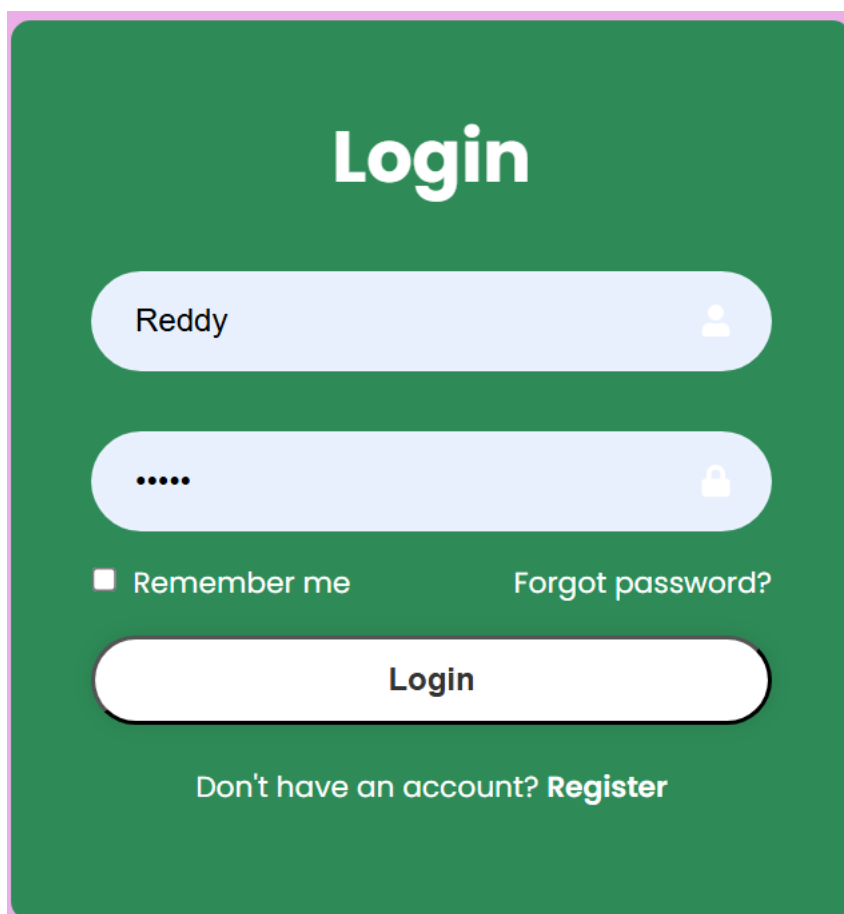
Table	Field	Description
User	id (PK)	Integer, Primary Key
	username	String(80), Unique, Not Null
	password_hash	String(256), Not Null
	first_name	String(100)
	last_name	String(100)
	email	String(120), Unique
	profile_pic	String(120), Default: 'default.png'
	role	String(50), Not Null, Default: 'user'
AlertHistory	id (PK)	Integer, Primary Key
	user_id (FK)	Integer, Foreign Key to User.id
	timestamp	DateTime, Default: UTC Now
	alert_type	String(50)
	message	String(255)
SystemSettings	id (PK)	Integer, Primary Key
	key	String(50), Unique, Not Null
	value	String(100)

Chapter 4

Results and System Functionality

This chapter showcases the final application, demonstrating its core features and functionality through screenshots of the user interface.

4.1 User Authentication

A screenshot of a login form with a green background. The word "Login" is centered at the top in white. Below it are two light blue rounded input fields. The first field contains the text "Reddy" and has a user icon on the right. The second field contains five dots and has a lock icon on the right. Below these fields are two links: "Remember me" with an unchecked checkbox, and "Forgot password?". At the bottom is a white rounded button with the text "Login". Below the button is the text "Don't have an account? Register" in white.

The system is protected by a secure login and registration system. Access to any analysis page requires authentication.

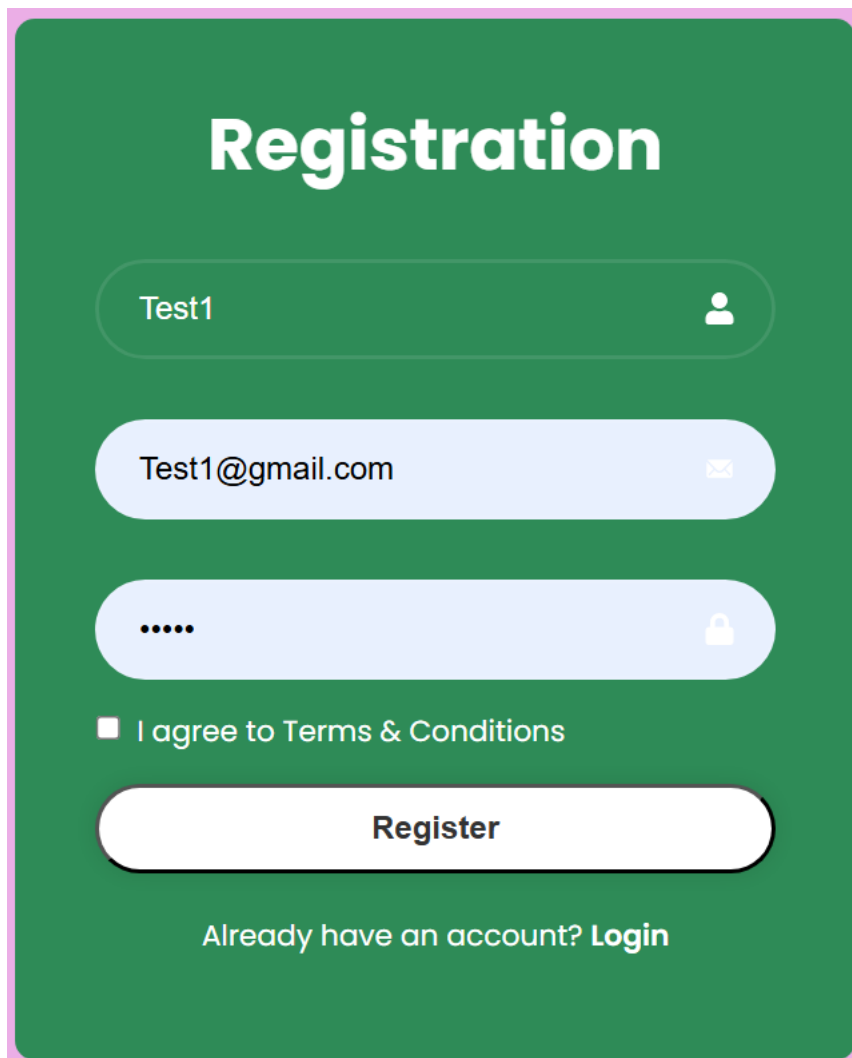
The image shows a user registration form on a green background. At the top, the word "Registration" is written in large white font. Below it are three light blue rounded input fields. The first field contains "Test1" and has a person icon on the right. The second field contains "Test1@gmail.com" and has an envelope icon on the right. The third field contains five dots "....." and has a lock icon on the right. Below these fields is a checkbox with the text "I agree to Terms & Conditions". At the bottom is a white rounded button with the text "Register". Below the button, the text "Already have an account? Login" is displayed in white.

Figure 4.2: User Registration Page

4.2 Analysis and Live Dashboard

After logging in, the user can start a new analysis from their webcam or by uploading a video file (Figure 4.3). The main dashboard (Figure 4.4) displays the live, annotated

video stream. Bounding boxes are color-coded (green for safe, red for in-zone), and alerts are overlaid directly on the video. Below the stream, a data table provides per-person details, including dwell times and alert status, with options to download individual PDF reports.

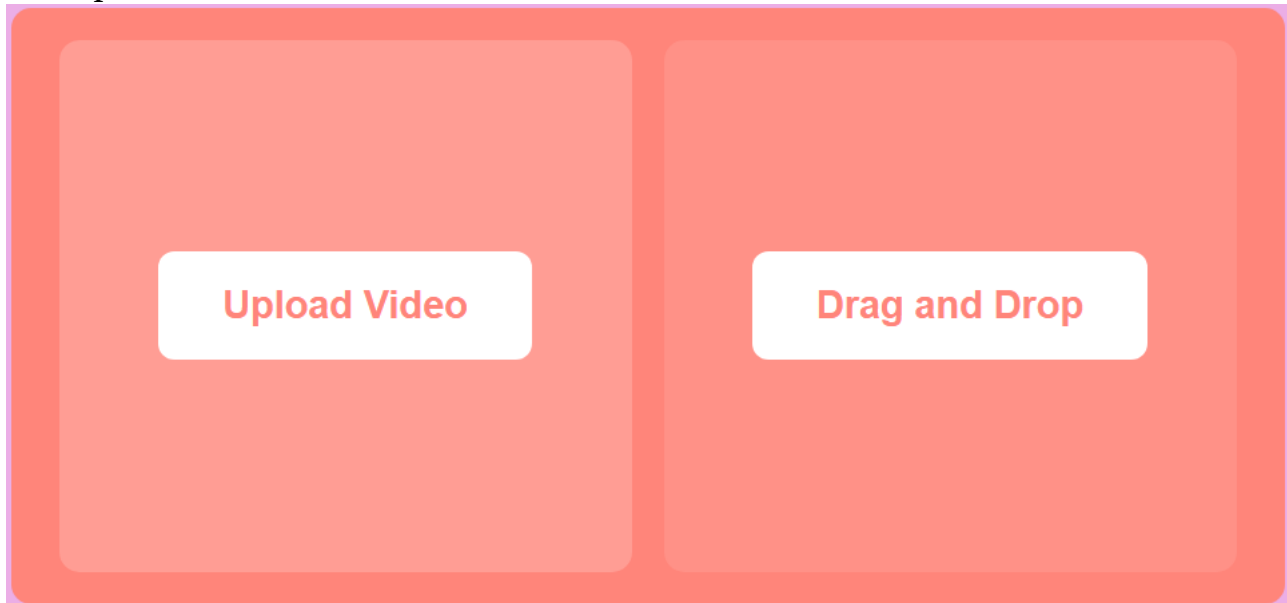


Figure 4.3: New Analysis Selection Page

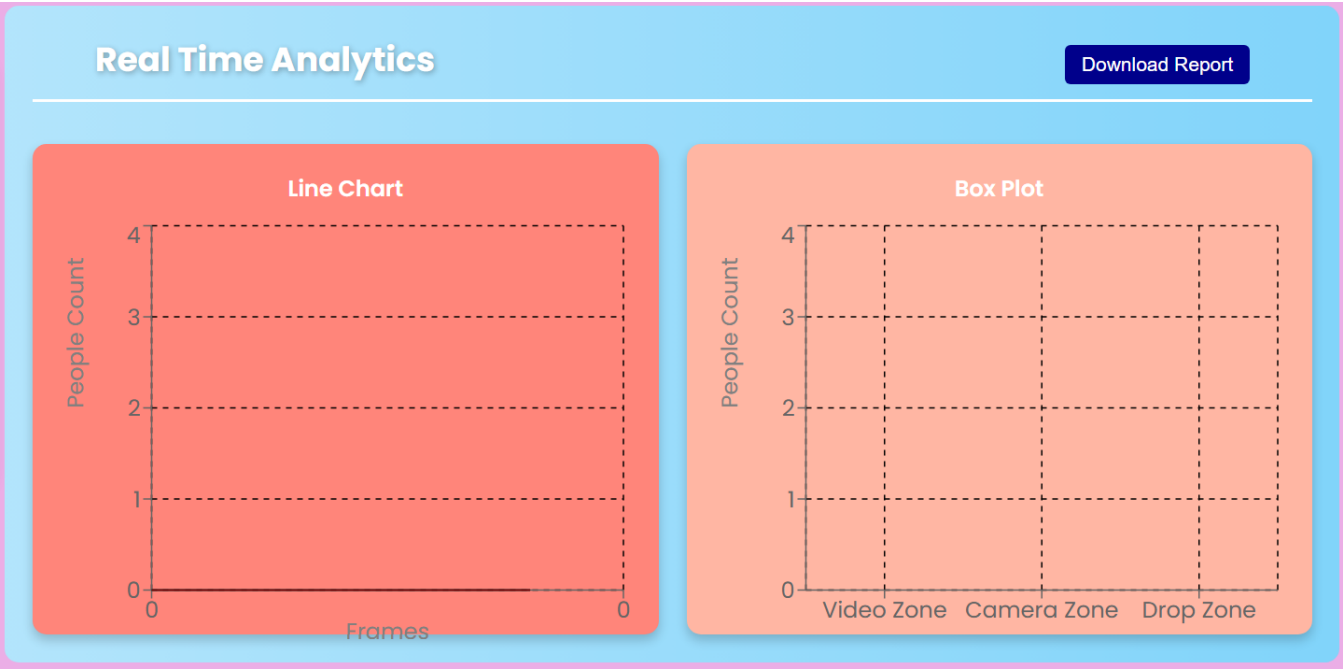


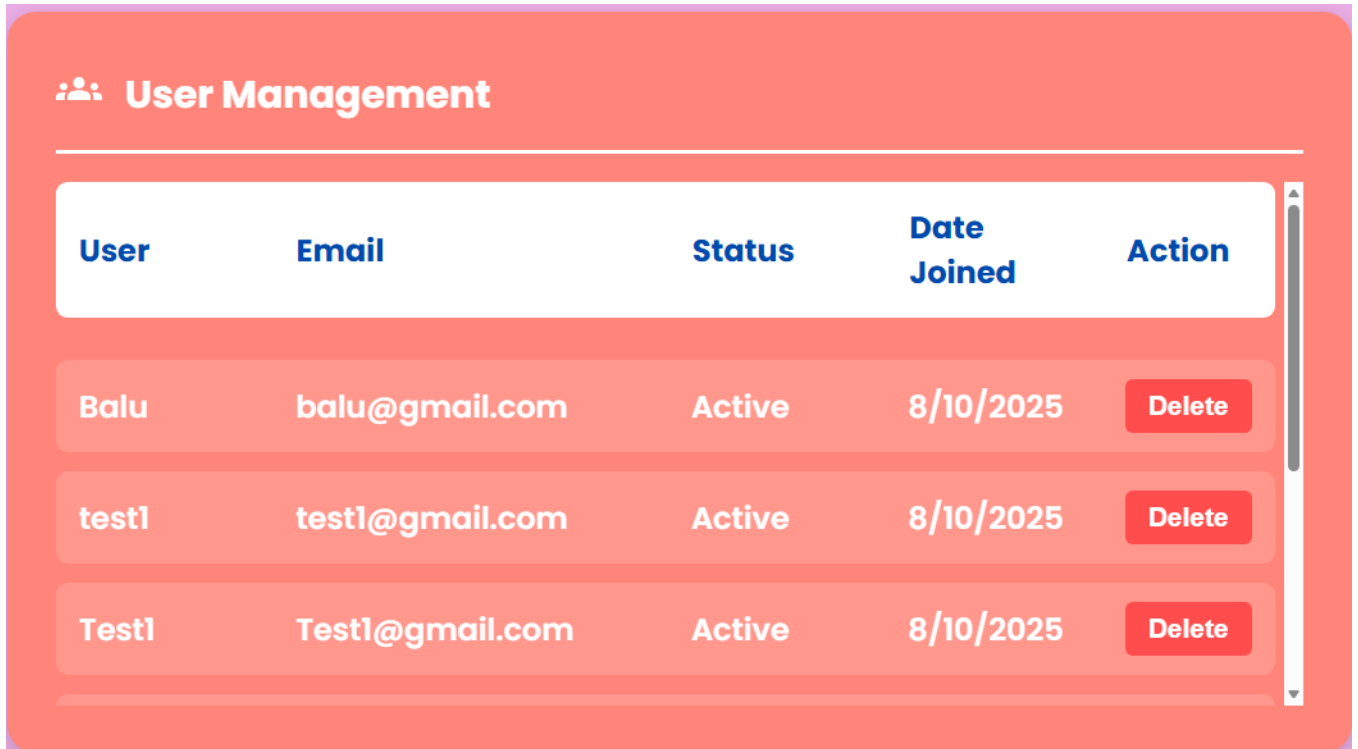
Figure 4.4: Live Dashboard Overview

4.3 Data Visualization and Reporting

The "Summary" page (Figure 4.5) provides live-updating charts, including a bar chart for Zone Population (Red vs. Green) and a line chart for Total Population Over Time. The "History" page (Figure 4.6) displays a clean, tabular log of all historical alerts for the user, with a master "Download History (CSV)" button.

4.4 Admin Panel and System Configuration

Users with the 'admin' role have access to a secure "Admin Panel" (Figure 4.7). This panel features a tabbed interface for:



User Management				
User	Email	Status	Date Joined	Action
Balu	balu@gmail.com	Active	8/10/2025	Delete
test1	test1@gmail.com	Active	8/10/2025	Delete
Test1	Test1@gmail.com	Active	8/10/2025	Delete

Figure 4.5: Live Summary Dashboard with Charts

- **User Management:** Promoting/demoting users and deleting accounts.
- **Global Settings:** Modifying the live alert thresholds for `person_threshold`, `zone_threshold`, and `overall_threshold`. Changes made here are saved to the database and instantly applied to the `detector.py` module without a server restart.
- **Global Activity Log:** Viewing the complete alert history for all.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Frames	People	CameraZonePeople	DropZonePeople	Timestamp									
2	17	1	184	13	6:02:27 PM									
3	22	4	184	13	11:09:27 AM									

Figure 4.6: Alert History Page with CSV Download.

Chapter 5

Challenges and Future Scope

5.1 Challenges Encountered

During development, two primary technical challenges were identified:

- **Object Occlusion:** In dense crowds, when individuals cross paths or are obscured by other people, the YOLO tracker occasionally swapped IDs or lost track of a person. While YOLO is robust, severe and prolonged occlusion remains a significant challenge in person tracking, leading to potential inaccuracies in dwell time calculation.
- **Inference Speed vs. Accuracy:** Achieving real-time performance (e.g., >25 FPS) is a trade-off with detection accuracy. The lightweight yolov8n.pt (nano) model was chosen for its high speed, but it can occasionally miss detections in complex scenes compared to larger models like yolov8m.pt (medium). This is a critical consideration for deployment on non-GPU hardware.

5.2 Future Enhancements

The current system provides a strong foundation. The following enhancements could significantly expand its capabilities:

- **Multi-Zone and Multi-Level Alerts:** Enhance the system to support drawing multiple, named zones (e.g., "Exit," "Restricted," "Queue"). These zones could have different alert levels, such as a 'Yellow' warning zone and a 'Red' danger zone.
- **External Alert Integration:** Integrate with third-party messaging services like

Twilio (for SMS) or SendGrid (for email). This would allow the system to send automatic, critical alerts to security personnel's mobile devices, rather than relying on dashboard monitoring.

- **Cloud Deployment and Multi-Camera View:** Re-architect the application for cloud deployment (e.g., on AWS or Azure) to ensure scalability. This would involve creating a centralized dashboard that can ingest and manage multiple camera streams simultaneously from different physical locations.

Edge AI Device Compatibility: Optimize the AI model and processing pipeline to run efficiently on low-power Edge AI devices (e.g., NVIDIA Jetson, Raspberry- Pi). This would allow for decentralized, on-site processing, reducing latency and network bandwidth requirements.

- **Advanced Analytics:** Implement more advanced analytics, such as anomaly detection (e.g., detecting falls or erratic movement) and predictive modeling to forecast overcrowding before it happens.

Chapter 6

Conclusion

This project successfully demonstrates the design, development, and deployment of a complete AI-Based Crowd Monitoring and Safety Management System. By integrating a high-performance YOLOv8 model with a robust Flask backend, PostgreSQL database, and a dynamic Chart.js dashboard, the project delivers a unified and automated solution for crowd analysis.

The system effectively replaces inefficient manual surveillance by providing:

- Real-time, reliable person detection and tracking with intelligent Danger Zone monitoring.
- A flexible 3-tier alert mechanism for per-person, zone, and overall population breaches.
- A secure, role-based platform featuring an advanced admin panel for live, dynamic system configuration.
- Actionable insights through a live analytics dashboard and exportable PDF/CSV reports for historical analysis.

The resulting application provides a strong and scalable foundation for real-world deployments in diverse fields, including public safety, retail analytics, event management, and operational monitoring. It showcases how modern AI and full-stack web technologies can be combined to build smart, scalable, and responsive safety systems.

Reference

- [1] Dense-stream YOLOv8n: A lightweight framework for real-time crowd counting and safety (<https://www.nature.com/articles/s41598-024-58937-z>)
- [2] YOLOv8 Object Tracking and Counting - LearnOpenCV (<https://learnopencv.com/yolov8-object-tracking-and-counting/>)
- [3] Optimize Safety with Smart Crowd Management Solutions - Viso.ai (<https://viso.ai/crowd/>)
- [4] Vision AI for crowd management - Ultralytics (<https://ultralytics.com/solutions/crowd-management/>)
- [5] Crowd Gathering Detection using YOLOv8 - GitHub (<https://github.com/ProRiko/Crowd-Gathering-Detection>)
- [6] Advanced Crowd and Event Management through AI - KE Leaders (<https://keleaders.com/advanced-crowd-event-management/>)
- [7] AI Crowd Management for Real-Time Monitoring (<https://iprogrammer.au/post/2756/ai-crowd-management-for-real-time-monitoring>)
- [8] How Spatial AI Is Transforming Crowd Management - Outsight (<https://insights.outsight.ai/how-spatial-ai-is-transforming-crowd-management-in-public-spaces/>)

Appendices

Appendix A

Setup and Installation Guide

This appendix provides the necessary steps to set up and run the project in a local development environment.

A.1 Prerequisites

- Python 3.8 or higher
- React
- Redis-server

A.2 Installation Steps

1. Clone the repository:

```
1 git clone https://github.com/your-username/crowd-monitoring-ai.git
2 cd crowd-monitoring-ai
```

2. Create and activate a virtual environment:

```
1 # Windows
2 python -m venv myenv
3 myenv\Scripts\activate
4
5 # macOS/ Linux
6 python3 -m venv myenv
7 source myenv/bin/activate
```

3. Install dependencies: Install all required packages from the requirement.txt file.

```
1 pip install -r requirement.txt
```

4. Install Redis-server: install redis-server from windows64 bit.

A.3 Running the Application

1. **Initialize the database:** The first time you run the application, it will create all the necessary tables based on the models defined in app.py.

2. **Run the Django server:**

```
1 python manage.py runserver
```

3. **Access the application:** Open a web browser and navigate to <http://127.0.0.1:8000>.

Note: The first user to register will be automatically assigned the 'admin' role. All subsequent users will be 'user' by default.

A.4 Project Dependencies (requirement.txt)

```
Django
React
Recharts
Django-restframework
Pandas
python - dotenv
opencv - python
ultralytics
Django-coreheader
numpy
```

